

LAPORAN PRAKTIKUM
PRAKTIKUM PEMROGRAMAN WEB
PERTEMUAN 7
STYLESHEET



Disusun oleh:

Nama : Reyhan Abelard Fikri
NIM : 25/562000/SV/26713
Kelas : TRPL PL1B2 2025
Dosen Pengampu : Achmad Choirudin Emcha, S.Kom., M.Eng.

PROGRAM STUDI D-IV TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026

PERTEMUAN 7

STYLESHEET

A. Tujuan Percobaan

1. Memahami penggunaan CSS *Position* seperti *relative*, *absolute*, *fixed*, dan *sticky* serta penerapan *z-index* dalam pengaturan tata letak halaman.
2. Menerapkan *Flexbox* untuk membuat *layout* yang rapi dan responsif menggunakan *justify-content*, *flex-wrap*, *gap*, dan *media query*.
3. Membuat tampilan web yang lebih interaktif dengan *transition*, *transform*, *@keyframes*, dan CSS Variables.

B. Dasar Teori

CSS *Position* digunakan untuk mengatur letak elemen pada halaman web. Nilai seperti *static*, *relative*, *absolute*, *fixed*, dan *sticky* memiliki fungsi yang berbeda. *Static* adalah posisi default, sedangkan *relative* memungkinkan elemen bergeser dari posisi awal tanpa mengubah susunan elemen lain. *Absolute* membuat elemen keluar dari alur normal dan mengikuti elemen induk terdekat yang memiliki *position* selain *static*. Properti ini sering digunakan untuk badge, ikon, atau komponen yang membutuhkan posisi presisi.

Selain itu, *fixed* dan *sticky* banyak digunakan pada navigasi modern. *Fixed* membuat elemen tetap berada pada posisi tertentu terhadap *viewport*, misalnya tombol *Back to Top*. Sementara *sticky* membuat elemen bergerak normal lalu menempel pada posisi tertentu saat halaman di-*scroll*. Untuk elemen yang saling bertumpuk, digunakan *z-index* agar urutan lapisan tampilan dapat diatur dengan jelas.

Flexbox adalah metode CSS yang digunakan untuk membuat tata letak lebih fleksibel dan responsif. Elemen induk disebut *flex container* dengan *display: flex*, sedangkan elemen anak disebut *flex item*. Properti seperti *justify-content*, *align-items*, *flex-wrap*, dan *gap* membantu menyusun elemen secara rapi tanpa perlu menggunakan *float*. Dengan tambahan *media query*, tampilan juga dapat menyesuaikan ukuran layar desktop maupun mobile.

Animation, *transition*, dan *transform* digunakan untuk membuat tampilan web lebih interaktif. *Transition* membuat perubahan properti berjalan halus, sedangkan *transform*

memungkinkan rotasi, pergeseran, atau pembesaran elemen. Untuk animasi yang lebih kompleks digunakan *@keyframes*, seperti efek *fade in*, *spinner*, dan *flip card*. CSS Variables juga membantu menjaga konsistensi warna dan ukuran karena nilai dapat digunakan berulang dari satu tempat yang sama.

C. Hasil dan Pembahasan

Pada bab ini dijelaskan hasil implementasi dari setiap tugas praktikum yang telah dikerjakan, mulai dari penggunaan CSS *Position*, *Flexbox*, hingga *Animation* dan *Transition*. Setiap soal dirancang untuk melatih pemahaman terhadap konsep *stylesheet* dalam pembuatan tampilan web yang terstruktur, responsif, dan interaktif. Selain itu, setiap bagian juga disertai penjelasan mengenai fungsi dari kode yang digunakan agar lebih mudah dipahami.

Pembahasan dilakukan secara bertahap pada tiga soal utama. Soal pertama berfokus pada pengaturan posisi elemen menggunakan berbagai nilai *position*, soal kedua membahas penyusunan *layout* responsif menggunakan *Flexbox*, dan soal ketiga menerapkan animasi CSS untuk meningkatkan interaktivitas tampilan. Dengan adanya pembahasan ini, diharapkan proses pembuatan halaman web tidak hanya dipahami dari sisi hasil akhir, tetapi juga dari logika dan fungsi setiap properti CSS yang digunakan.

1. Latihan Nomor 1

Pada soal pertama, digunakan beberapa jenis CSS *Position* yaitu *sticky*, *relative*, *absolute*, dan *fixed* untuk mengatur tata letak halaman. Navbar dibuat dengan *position: sticky* dan *top: 0* agar tetap berada di bagian atas saat halaman di-*scroll*. Penggunaan *z-index* memastikan navbar selalu tampil di depan sehingga tidak tertutup elemen lain. Cara ini membuat navigasi lebih mudah diakses oleh pengguna selama membuka halaman.

Profile card menggunakan *position: relative* sebagai acuan bagi elemen anak yang memakai *position: absolute*, yaitu badge status yang diletakkan di sudut kanan atas card. Tombol *Back to Top* menggunakan *position: fixed* sehingga tetap berada di kanan bawah layar meskipun halaman digulir. Dari tugas ini dapat dipahami bahwa setiap jenis *position* memiliki fungsi berbeda dan saling melengkapi dalam membangun tampilan web yang rapi dan nyaman digunakan.

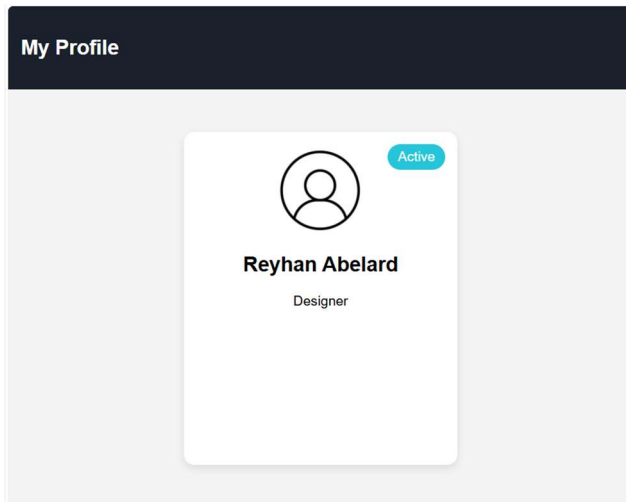
Kode HTML :

```
1  <!DOCTYPE html>
2  <html lang="id">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Tugas 1</title>
7      <link rel="stylesheet" href="latsol1.css">
8  </head>
9  <body>
10
11      <nav class="navbar">
12          <h2>My Profile</h2>
13      </nav>
14
15      <section class="container">
16          <div class="profile-card">
17              <div class="badge-status">Active</div>
18              ✨ 
19              <h2>Reyhan Abelard</h2>
20              <p>Designer</p>
21          </div>
22      </section>
23
24      <button class="back-to-top" onclick="window.scrollTo({top:0, behavior:'smooth'})">
25          ↑
26      </button>
27
28  </body>
29  </html>
```

Kode CSS :

```
1  :root {
2    --primary: #5c6bc0;
3    --accent: #26c6da;
4    --nav-bg: #1a202c;
5  }
6
7  body {
8    margin: 0;
9    font-family: Arial;
10   background: #f4f4f4;
11 }
12
13 .navbar {
14   position: sticky;
15   top: 0;
16   background: var(--nav-bg);
17   color: white;
18   padding: 15px;
19   z-index: 100;
20 }
21
22 .container {
23   display: flex;
24   justify-content: center;
25   padding: 50px;
26 }
27
28 .profile-card {
29   position: relative;
30   width: 280px;
31   height: 350px;
32   background: white;
33   border-radius: 12px;
34   text-align: center;
35   padding: 20px;
36   box-shadow: 0 4px 10px rgba(0,0,0,0.1);
37 }
38
39 .profile-card img {
40   width: 100px;
41   border-radius: 50%;
42 }
43
44 .badge-status {
45   position: absolute;
46   top: 14px;
47   right: 14px;
48   background: var(--accent);
49   color: white;
50   padding: 6px 12px;
51   border-radius: 20px;
52   z-index: 10;
53 }
54
55 .back-to-top {
56   position: fixed;
57   bottom: 28px;
58   right: 28px;
59   padding: 15px;
60   border: none;
61   border-radius: 50%;
62   background: var(--primary);
63   color: white;
64   font-size: 20px;
65   cursor: pointer;
66   z-index: 999;
67 }
```

Output :



2. Latihan Nomor 2

Pada soal kedua, digunakan konsep *Flexbox* untuk membuat tata letak halaman yang rapi dan responsif. Navbar dibuat dengan *display: flex* dan *justify-content: space-between* sehingga logo otomatis berada di sisi kiri dan menu navigasi berada di sisi kanan tanpa perlu menggunakan *float*. Properti *align-items: center* membantu menjaga posisi elemen tetap sejajar secara vertikal. Pendekatan ini membuat kode lebih sederhana dan mudah dipelihara dibandingkan metode layout lama.

Bagian *hero section*, *card grid*, dan *footer* juga menggunakan *Flexbox* untuk mengatur distribusi ruang secara efisien. *Hero section* menampilkan teks dan gambar secara sejajar, sedangkan *card grid* memanfaatkan *flex-wrap: wrap* agar card dapat turun ke baris berikutnya saat ruang tidak mencukupi. Footer dibuat dalam tiga kolom menggunakan *gap* agar jarak antar elemen tetap konsisten. Dengan tambahan *media query*, seluruh layout dapat berubah menjadi susunan vertikal pada layar kecil sehingga tampilan tetap nyaman digunakan pada perangkat mobile.

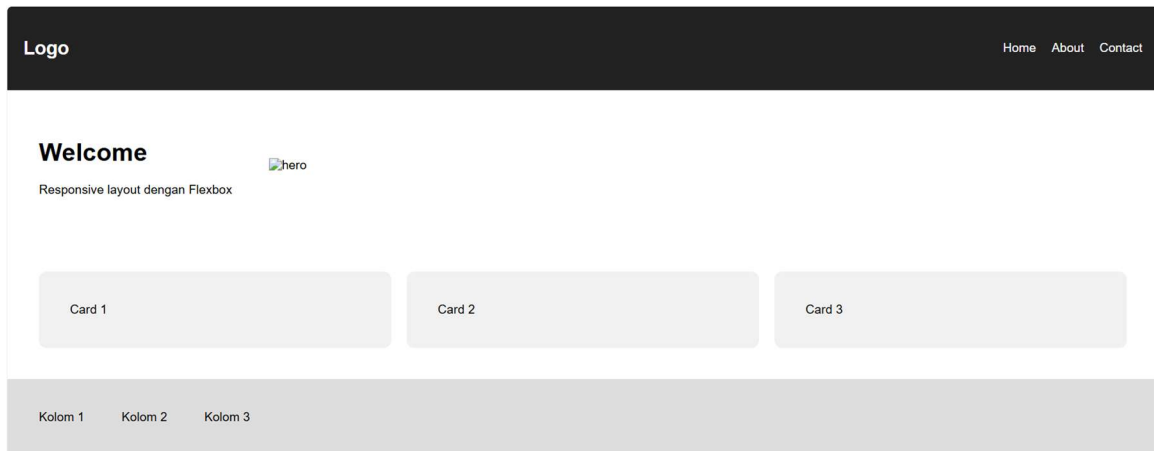
Kode HTML :

```
1 <!DOCTYPE html>
2 <html lang="id">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Tugas 2</title>
7   <link rel="stylesheet" href="latsol2.css">
8 </head>
9 <body>
10
11 <nav class="navbar">
12   <h2>Logo</h2>
13   <ul>
14     <li>Home</li>
15     <li>About</li>
16     <li>Contact</li>
17   </ul>
18 </nav>
19
20 <section class="hero">
21   <div>
22     <h1>Welcome</h1>
23     <p>Responsive layout dengan Flexbox</p>
24   </div>
25   
26 </section>
27
28 <section class="card-grid">
29   <div class="card">Card 1</div>
30   <div class="card">Card 2</div>
31   <div class="card">Card 3</div>
32 </section>
33
34 <footer class="footer">
35   <div>Kolom 1</div>
36   <div>Kolom 2</div>
37   <div>Kolom 3</div>
38 </footer>
39
40 </body>
41 </html>
```

Kode CSS :

```
1  body {
2      margin: 0;
3      font-family: Arial;
4  }
5
6  .navbar {
7      display: flex;
8      justify-content: space-between;
9      align-items: center;
10     padding: 20px;
11     background: #222;
12     color: white;
13 }
14
15 .navbar ul {
16     display: flex;
17     gap: 20px;
18     list-style: none;
19 }
20
21 .hero {
22     display: flex;
23     gap: 48px;
24     align-items: center;
25     padding: 40px;
26 }
27
28 .card-grid {
29     display: flex;
30     flex-wrap: wrap;
31     gap: 20px;
32     padding: 40px;
33 }
34
35 .card {
36     flex: 1 1 260px;
37     padding: 40px;
38     background: #f1f1f1;
39     border-radius: 10px;
40 }
41
42 .footer {
43     display: flex;
44     gap: 48px;
45     padding: 40px;
46     background: #ddd;
47 }
48
49 @media (max-width: 768px) {
50     .hero,
51     .card-grid,
52     .footer {
53         flex-direction: column;
54     }
55 }
```

Output :



3. Latihan Nomor 3

Pada soal ketiga, pembahasan berfokus pada penggunaan *animation*, *transition*, *transform*, dan CSS Variables untuk membuat tampilan yang lebih interaktif. Variabel warna diletakkan pada `:root` agar seluruh elemen dapat menggunakan nilai yang sama secara konsisten. Efek *fade in* dibuat menggunakan `@keyframes` sehingga halaman terlihat muncul secara halus saat pertama kali dibuka. Teknik ini memberikan kesan modern dan meningkatkan pengalaman visual pengguna saat mengakses halaman.

Selain itu, tombol diberikan efek *hover* menggunakan *transition* dan *transform* agar terasa lebih responsif ketika disentuh kursor. *Loading spinner* dibuat dengan animasi putaran menggunakan `@keyframes spin` yang berjalan terus-menerus sebagai indikator proses loading. Bonus *flip card* menggunakan `transform: rotateY()` untuk menciptakan efek kartu berputar saat diarahkan kursor. Seluruh penerapan ini menunjukkan bahwa CSS tidak hanya digunakan untuk mempercantik tampilan, tetapi juga meningkatkan interaksi dan kenyamanan pengguna secara keseluruhan.

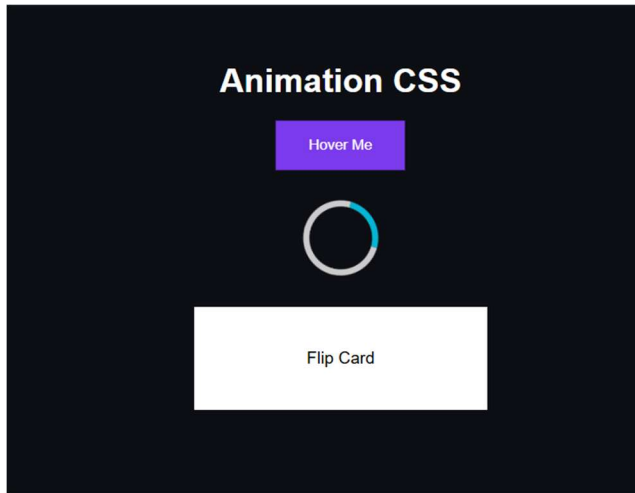
Kode HTML :

```
1  <!DOCTYPE html>
2  <html lang="id">
3  <head>
4  |   <meta charset="UTF-8">
5  |   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6  |   <title>Tugas 3</title>
7  |   <link rel="stylesheet" href="latsol3.css">
8  </head>
9  <body>
10
11 | <div class="page-wrapper">
12 | | <h1>Animation CSS</h1>
13 | |
14 | | <button class="btn">Hover Me</button>
15 | |
16 | | <div class="spinner"></div>
17 | |
18 | | <div class="flip-card">Flip Card</div>
19 | </div>
20
21 </body>
22 </html>
```

Kode CSS :

```
1  √ :root {
2    --primary: #7c3aed;
3    --accent: #06b6d4;
4    --bg: #0d0f14;
5  }
6
7  √ body {
8    margin: 0;
9    font-family: Arial;
10   background: var(--bg);
11   color: white;
12   text-align: center;
13 }
14
15 √ .page-wrapper {
16   padding: 50px;
17   animation: fadeInUp 0.7s ease;
18 }
19
20 √ @keyframes fadeInUp {
21   √ from {
22     opacity: 0;
23     transform: translateY(20px);
24   }
25   √ to {
26     opacity: 1;
27     transform: translateY(0);
28   }
29 }
30
31 √ .btn {
32   padding: 15px 30px;
33   border: none;
34   background: var(--primary);
35   color: white;
36   cursor: pointer;
37 }
38
39 √ .btn:hover {
40   transform: scale(1.05);
41   transition: 0.3s;
42 }
43
44 √ .spinner {
45   width: 60px;
46   height: 60px;
47   border: 6px solid #ccc;
48   border-top: 6px solid var(--accent);
49   border-radius: 50%;
50   margin: 30px auto;
51   animation: spin 1s linear infinite;
52 }
53
54 √ @keyframes spin {
55   from { transform: rotate(0deg); }
56   to { transform: rotate(360deg); }
57 }
58
59 √ .flip-card {
60   width: 200px;
61   margin: auto;
62   padding: 40px;
63   background: white;
64   color: black;
65   transition: 0.6s;
66 }
67
68 √ .flip-card:hover {
69   transform: rotateY(180deg);
70 }
```

Output :



D. Kesimpulan

Praktikum ini membantu memahami penggunaan CSS *Position*, *Flexbox*, serta *Animation* dalam pengembangan tampilan web. *Position* berguna untuk mengatur letak elemen secara spesifik, terutama pada *navbar*, *card*, dan tombol tetap. *Flexbox* mempermudah pembuatan *layout* yang rapi dan responsif tanpa kode yang rumit. Sementara itu, *animation* dan *transition* membuat tampilan lebih menarik dan interaktif. CSS Variables juga sangat membantu dalam menjaga konsistensi desain karena nilai dapat digunakan berulang dengan lebih efisien.